National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	15
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	MongoDB Data Manipulation
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

1 Updating Documents....1
1.1 Common Update Operators....1
1.2 Update One....2
1.3 Update Many....3
2 Deleting Documents....4
2.1 Delete One....5
2.2 Delete Many....5

1 Updating Documents

Updating documents in MongoDB is done using two main methods:

- `updateOne()`
- `updateMany()`

These methods take two parameters: a **filter document** (`{}`) to select records and an **update document** (`{}`) that specifies the changes using operators.

Updates can modify specific fields, arrays, and embedded documents without replacing the entire record.

General Syntax:

```
db.collection.updateOne(filter, update)

db.collection.updateMany(filter, update)
```

Examples:

```
// Update a single student's CGPA
db.students.updateOne(
    { name: "Ahmed" },
    { $set: { cgpa: 3.4 } }
)

// Update multiple students from SE department
db.students.updateMany(
    { department: "SE" },
    { $inc: { age: 1 } }
)
```

1.1 Common Update Operators

MongoDB provides operators to modify documents efficiently without replacing the whole document. These operators help update specific fields, arrays, or perform calculations dynamically.

Operator	Description
`$unset`	Removes a field from a document
`$set`	Sets or updates a field value
`$inc`	Increments/decrements numeric values
`$mul`	Multiplies (or divides using fractional values) numeric fields
`$push`	Adds an item to an array
`$pull`	Removes matching items from an array

1.2 Update One

`updateOne()` updates the first document that matches the filter. It is useful when you only want to modify a specific record without affecting others.

General Syntax:

```
db.collection.updateOne( {...}, {...} )
```

Examples:

```
// Remove address field
db.students.updateOne(
    { name: "Ali" },
    { $unset: { address: "" } }
)

// Update CGPA
db.students.updateOne(
    { name: "Ali" },
    { $set: { cgpa: 3.7 } }
)

// Increment age
db.students.updateOne(
    { name: "Ali" },
    { $inc: { age: 1 } }
```

```
)

// Increase CGPA by 25%
db.students.updateOne(
   { name: "Ali" },
   { $mul: { cgpa: 1.25 } }
)

// Add new skill
db.students.updateOne(
    { name: "Ali" },
    { $push: { skills: "MongoDB" } }
)

// Remove a skill
db.students.updateOne(
    { name: "Ali" },
    { $pull: { skills: "C++" } }
)

// Update embedded document (city)
db.students.updateOne(
    { name: "Ahmed" },
    { $set: { "address.city": "Rawalpindi" } }
)

// Update marks inside embedded array
db.students.updateOne(
    { name: "Ali", "courses.name": "DB" },
    { $set: { "courses.$.marks": 90 } }
)
```

1.3 Update Many

`updateMany()` modifies all documents that match the filter condition. It is useful for bulk updates such as department-wide changes or applying policy updates.

General Syntax:

```
db.students.updateMany( {...}, {...} )
```

Examples:

```
// Increase CGPA for all CS students
db.students.updateMany(
    { department: "CS" },
    { $inc: { cgpa: 0.1 } }
)

// Add a new skill to all SE students
db.students.updateMany(
    { department: "SE" },
    { $push: { skills: "Git" } }
)

// Remove zip code from all students in Lahore
db.students.updateMany(
    { "address.city": "Lahore" },
    { $unset: { "address.zip": "" } }
)

// Add a new course to all IT students
db.students.updateMany(
    { department: "IT" },
    { $push: { courses: { name: "Security", marks: 70 } } }
)
```

2 Deleting Documents

Deleting documents in MongoDB is performed using `deleteOne()` and `deleteMany()`. Both methods accept a **filter document** that defines which records should be removed.

General Syntax:

```
db.collection.deleteOne(filter)

db.collection.deleteMany(filter)
```

Examples:

```
// Delete a student with missing department
db.students.deleteOne({ department: null })

// Delete students with no skills
db.students.deleteMany({ skills: { $size: 0 } })
```

2.1 Delete One

`deleteOne()` removes only the first document that matches the filter. It is useful when deleting a specific record such as a student by name or ID.

General Syntax:

```
db.collection.deleteOne( {...} )
```

Examples:

```
// Delete student Ali
db.students.deleteOne({ name: "Ali" })

// Delete a student from Karachi
db.students.deleteOne({ "address.city": "Karachi" })
```

2.2 Delete Many

`deleteMany()` removes all documents that match the condition. It is useful for cleaning up bulk data such as low CGPA students or incomplete records.

General Syntax:

```
db.collection.deleteMany( {...} )
```

Examples:

```
// Delete students with low CGPA
db.students.deleteMany({ cgpa: { $lt: 2.5 } })
```

```
// Delete students who have OS marks less than 65
db.students.deleteMany({ "courses.marks": { $lt: 65 } })
```